

Breve manual de diseño de la práctica del Sistema Automático de Venta de Billetes de Cercanías (SAVBC)

Tabla de contenidos

1- Introducción.....	2
2- Clases propuestas.....	2
3- Operativa general del kiosco.....	6
3.1 Inicio de la aplicación y bucle principal.....	6
3.2 Operativa de carrusel.....	7
3.3 Pantalla de pago.....	10
4- Conclusiones.....	11

Versión 1.0.21

17:02 - 02/12/25

1 Introducción

El siguiente documento pretende guiar al estudiante por un diseño adecuado para implementar la práctica del Sistema Automático de Venta de Billetes de Cercanías (SAVBC).

El diagrama de actividad de la Figura 1 muestra a grandes rasgos la lógica asociada a la aplicación SAVBC. Obsérvese que este diagrama no contempla la lógica de bajo nivel asociada a cada operación. Este diagrama tampoco refleja la estructura de clases del programa.

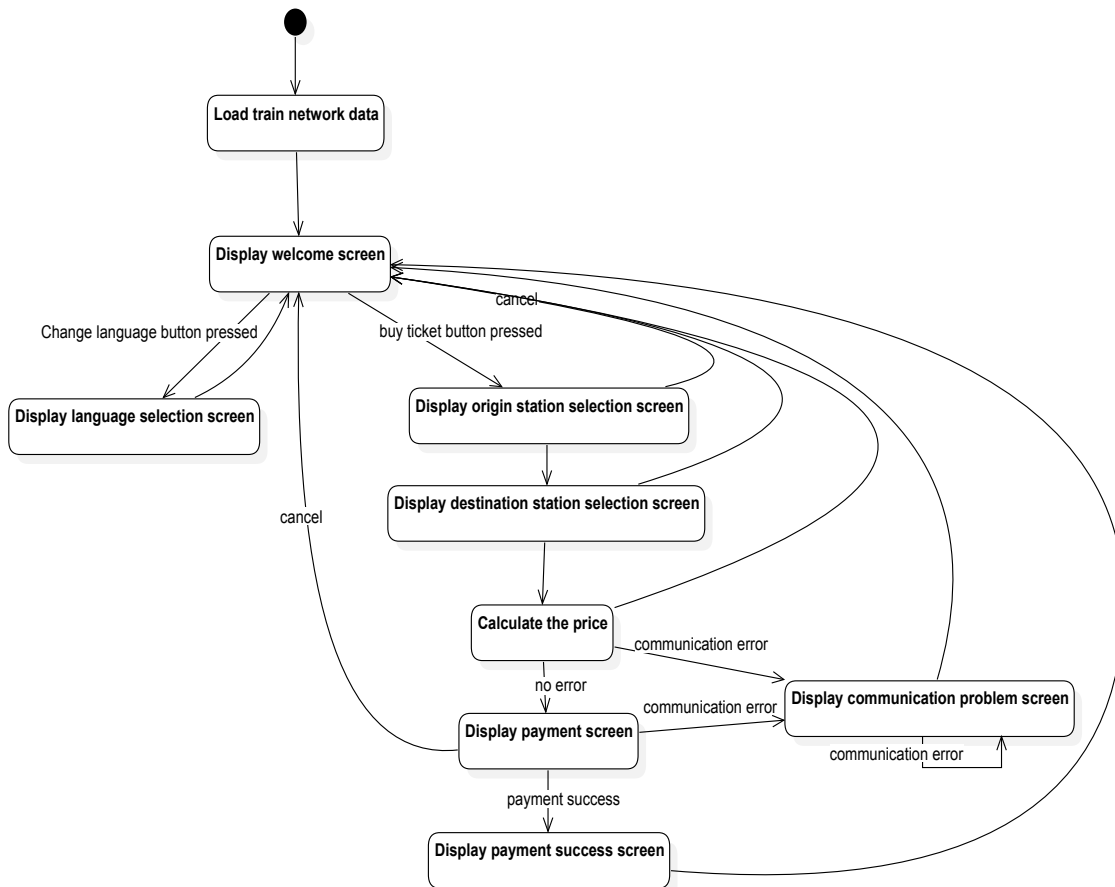


Figura 1.- Diagrama de actividad con la lógica de alto nivel asociada a la práctica.

2 Clases propuestas

La Figura 2 presenta las clases principales que gestionan la operativa. A continuación se resume la responsabilidad de cada clase de este diagrama.

- `CommuterTicketDispenser`.- Encargada de iniciar el programa.
- `TicketDispenserManager`.- Encargada de iniciar los diferentes elementos que componen la aplicación y de contener el bucle de control que va mostrando las sucesivas pantallas.

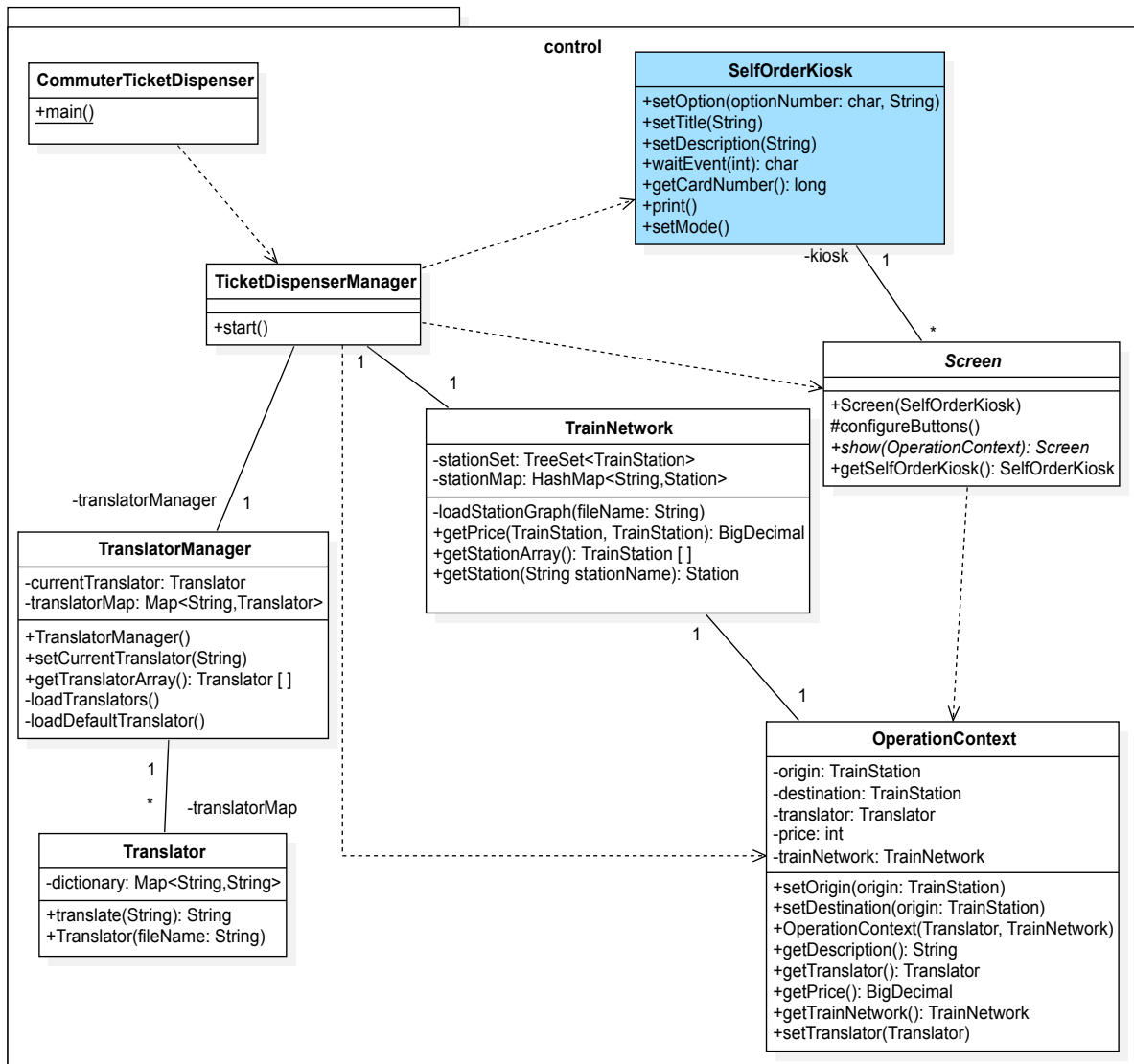


Figura 2.- Diagrama de clases principal.

- **CommuterTicketDispenser.**- Encargada de iniciar el programa.
- **TranslatorManager.**- Encargada de cargar, almacenar y proporcionar los traductores a los diferentes idiomas.
- **Translator.**- Encargada de traducir de un idioma.
- **KioskScreen.**- Clase base de todas las pantallas. El método `configureButtons` borra todos los botones, de manera que las clases derivadas puedan llamarlo y reimplementarlo.
- **OperationContext.**- Encargada de almacenar los detalles de una operación mientras está en curso. Las diferentes pantallas se van pasando un objeto de este tipo para ir rellenándolo.
- **TrainNetwork.**- Carga, almacena y proporciona las estaciones y el precio.

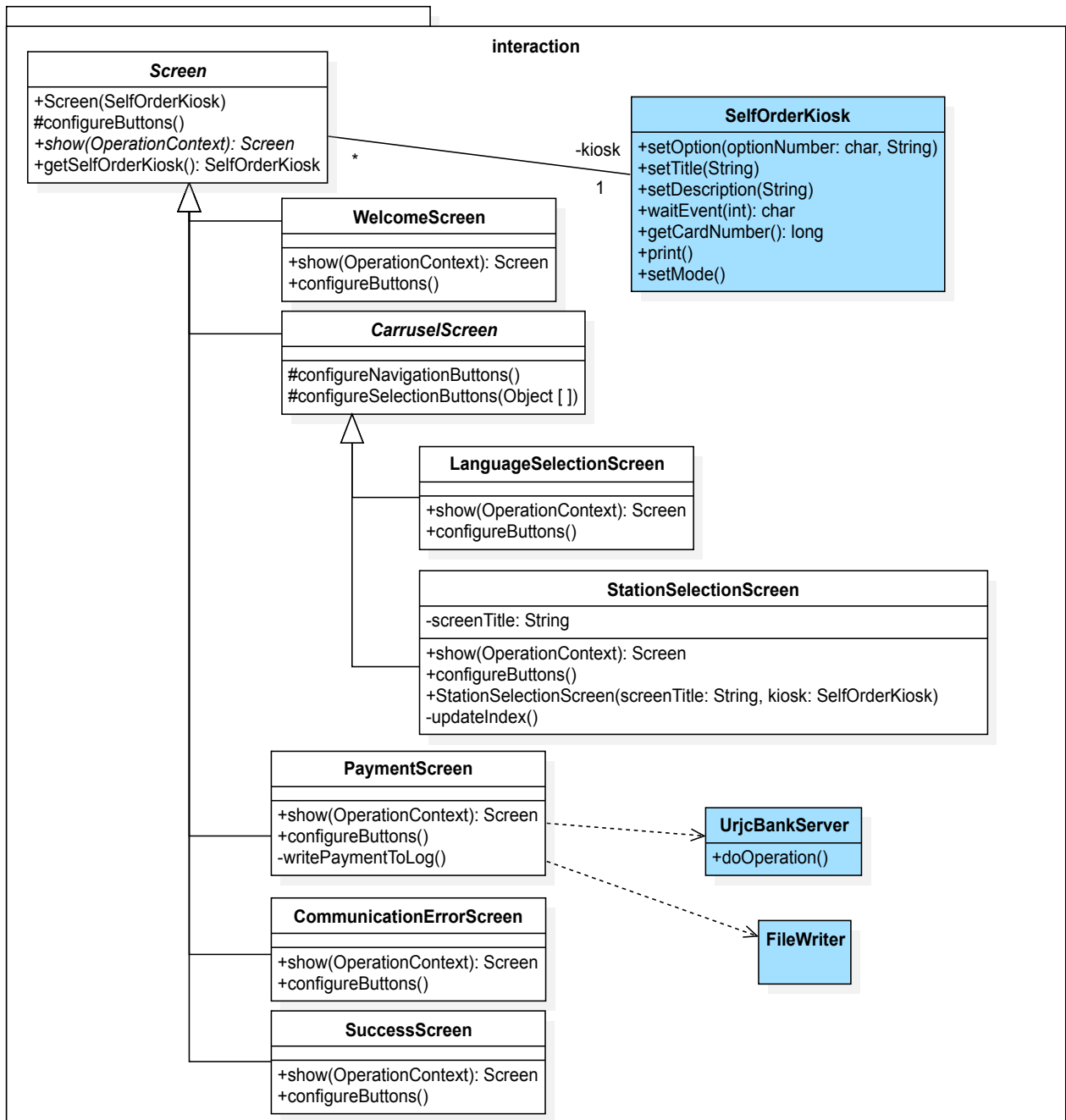


Figura 3.- Diagrama de clases que refleja los modelos utilizados por las diferentes pantallas con las que interactuará el usuario.

La Figura 3 presenta las clases que derivan de `KioskScreen` y que se encargan de modelar las diferentes pantallas. A continuación se resume la responsabilidad de cada clase de esta figura.

- `WelcomeScreen`. – Gestiona la pantalla que ve el usuario al inicio de su interacción. Dicha pantalla mostrará dos botones: cambiar idioma o iniciar compra.
- `CarruselScreen`. – Clase abstracta de la que heredan las clases que necesitan navegación, como idioma o selección de estación. Tiene un par de métodos protegidos que

configuran los botones de navegación o los botones con las opciones. Las clases que deriven podrán usarlos.

- `LanguageSelectionScreen`. – Gestiona la pantalla de selección del idioma.
- `StationSelectionScreen`. – Gestiona la pantalla de selección de estación. Esta clase se podrá usar para la estación de origen y la de destino, es por eso que en el constructor se le pasa en título de la ventana.
- `CommuinicationErrorScreen`. – Gestiona la pantalla de error de comunicación. Mientras se presenta esta pantalla el sistema comprueba cada 5 segundos si ya ha vuelto la comunicación con el banco, en cuyo caso se vuelve a la pantalla `WelcomeScreen`. También puede pedir que se retire la tarjeta si la detecta.
- `SuccessScreen`. – Gestiona la pantalla que ve el usuario para indicarle que la operación ha sido un éxito, y para pedirle que retire la tarjeta y el ticket.

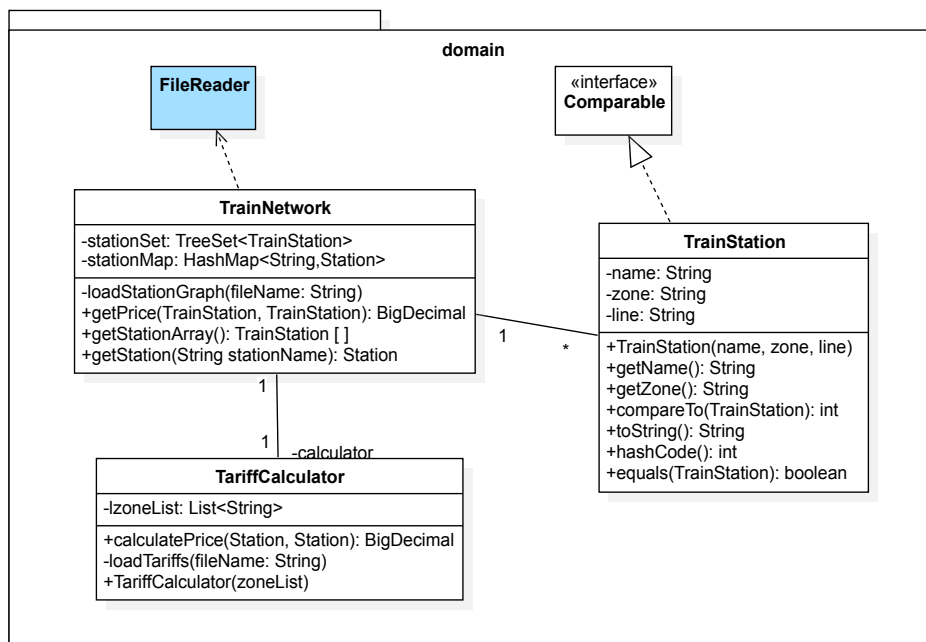


Figura 4.- Diagrama de clases para las estaciones y los precios.

Finalmente, la Figura 4 presenta las clases relacionadas con la información sobre la red de trenes y sus tarifas. Las clases de este diagrama ya han sido presentadas, excepto `TrainStation`.

- `TrainStation`.- Modela la información sobre una estación de tren.
- `TariffCalculator`.- Esta clase carga de disco el precio de las tarifas y calcula el precio entre dos estaciones, teniendo en cuenta el número de zonas que se atraviesa. Por ejemplo, de “Alcorcón Central” a “Móstoles Central” se atraviesa una zona, pero de Móstoles (B2) a Vicálvaro (B2) se atraviesan 4 zonas. Para ello, esta clase usará la siguiente lógica simplificada:
 - Si dos estaciones están en la misma línea, el número de zonas atravesadas será igual a la distancia entre ambas zonas en la lista de zonas ordenadas lexicográficamente. Por

ejemplo, si las estaciones ordenadas en una lista son ["A", "B1", "B2", "B3", "C1", "C2", "E1", "Zona Verde"] y se desea ir de la estación de "Alcorcón Central", que es zona B1 y línea C5, a la de "Móstoles Central" que es C-5 y B2, las zonas atravesadas serán 1, que corresponde a la distancia entre las cadenas "B1" y "B2" en la lista. Si se desea ir de "Alcorcón" a "Las Retamas", ambas de la zona B1 y en la misma línea, el número de zonas atravesadas es 0.

- En caso de que estén en diferente línea, el número de zonas atravesada se calculará como la suma de las distancias entre cada zona y la primera zona de la lista. Por ejemplo, si las estaciones ordenadas en la lista son ["A", "B1", "B2", "B3", "C1", "C2", "E1", "Zona Verde"] y se desea ir de la estación de "Alcorcón Central", que es zona B1 y línea C5, a la de "Parla" que es línea C-3 y zona B2, las zonas atravesadas serán 3 (la suma de la distancia entre "B1" y "A" en la lista, que es 1, y la suma de la distancia entre "A" y "B2" en la lista, que es 2).
- Si la suma de zonas supera el valor máximo del fichero, se pondrá la tarifa máxima.

3 Operativa general del kiosco

Partiendo de estas clases, el Diagrama de Actividad de la Figura 1 quedaría segmentado en las clases y métodos que se ven el diagrama de la Figura 5.

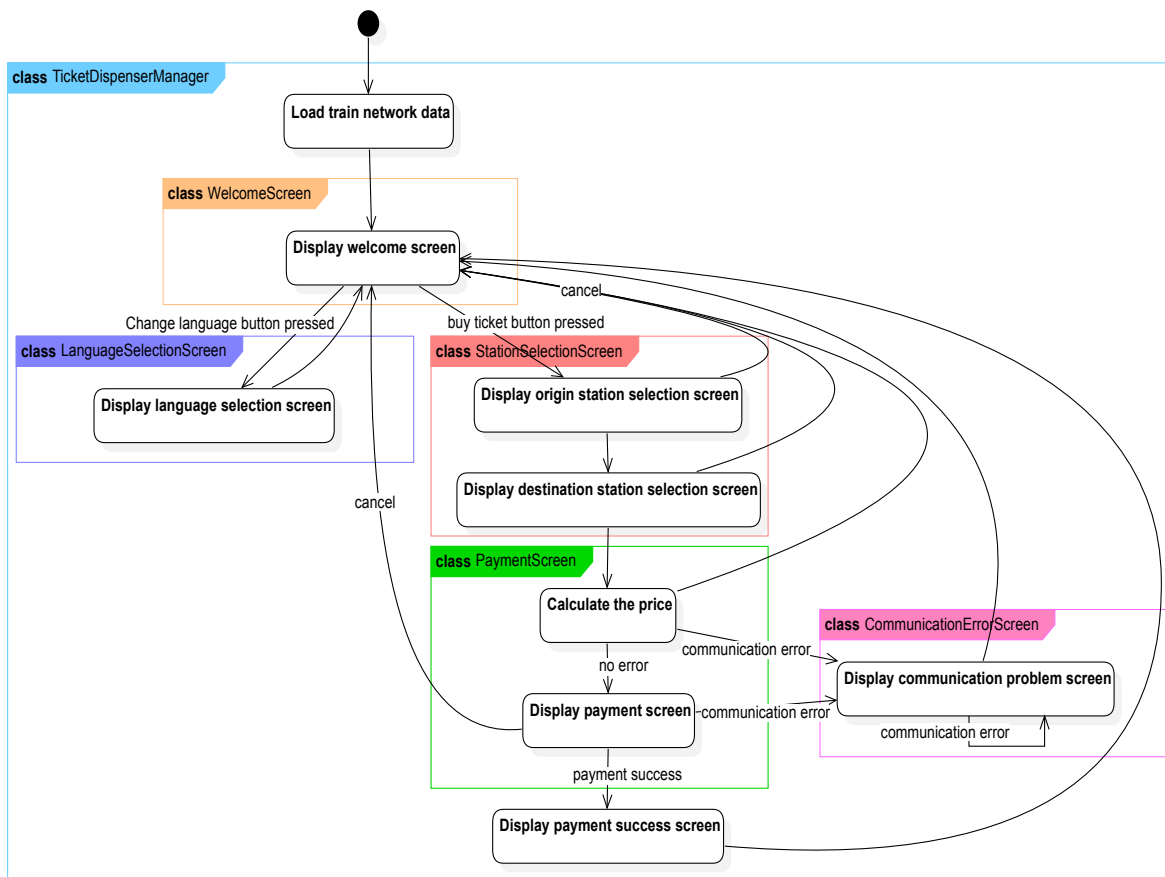


Figura 5.- Diagrama de actividad con la lógica de alto nivel asociada a la práctica repartida en las clases que lo implementarían.

3.1 Inicio de la aplicación y bucle principal

El diagrama de secuencia de la Figura 6 ilustra el inicio de la aplicación.

En el diagrama se ve que al inicio de la aplicación se crea el `TicketDispenseManager` que a su vez crea un objeto `TrainNetwork`. Este objeto carga las estaciones y las tarifas. Luego, se crean objetos para los idiomas, el `SelfOrderKiosk` y el `OperationContext`. Finalmente, se crea la pantalla de bienvenida y la sitúa como pantalla actual.

Finalmente, entra en un bucle que muestra la pantalla actual. Cuando finaliza la ejecución de la pantalla actual, dicha pantalla devolverá la siguiente pantalla que deberá calificarse como pantalla actual. Así, el orden de las pantallas viene determinado por las propias pantallas. Si se quisiese añadir una pantalla hay que añadirlo solo en la pantalla que le dé paso.

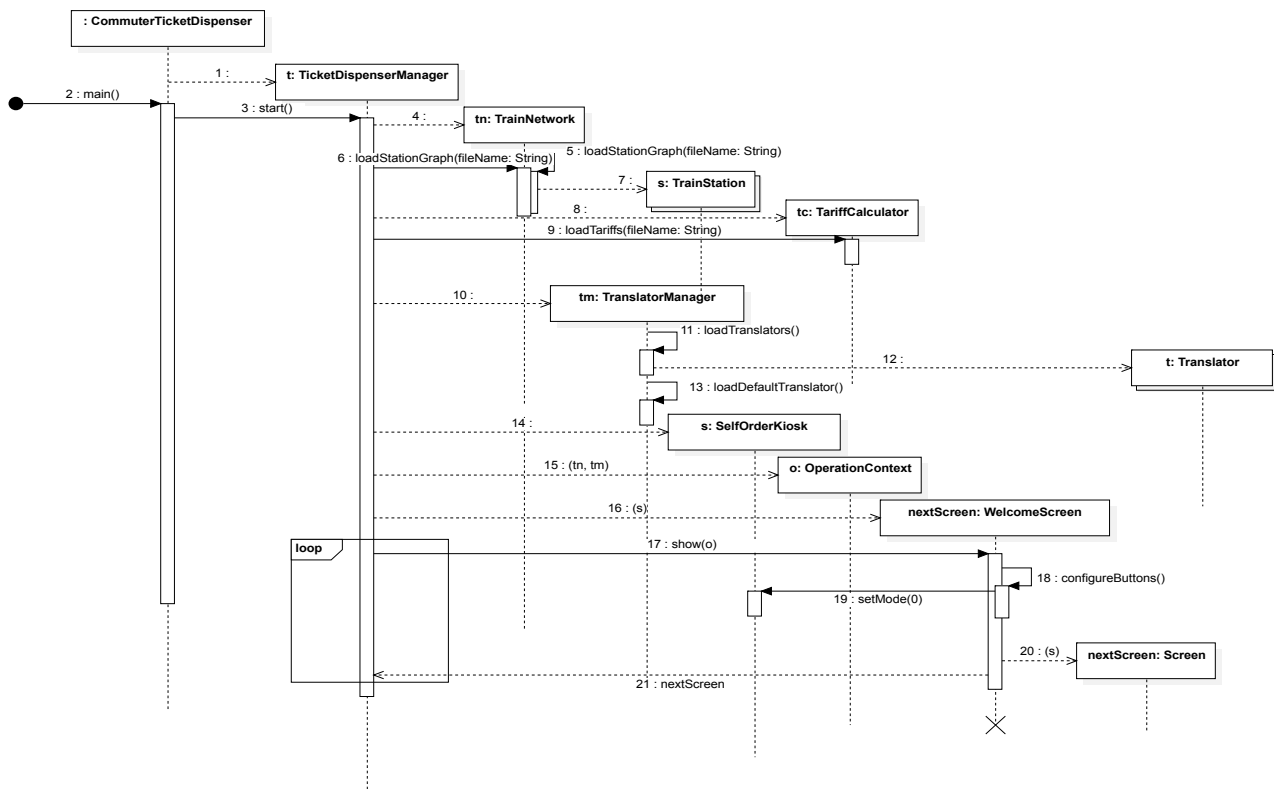


Figura 6.- Diagrama de secuencia que refleja el inicio del programa.

3.2 Operativa de carrusel

Los diagramas de las Figuras 7, 9 y 10 ilustran las operaciones que ocurren dentro de la pantalla del carrusel cuando se les pide que muestren una lista de elementos.

En general, el carrusel es una pantalla que se debe usar cuando el número de elementos a mostrar excede el número de botones del kiosco.

Respecto a la implementación se puede decir que todas las pantallas de tipo carrusel derivan de la clase `CarouselScreen`. En esta clase padre se podrán poner, además de los métodos que se indican, todos los métodos comunes que se considere oportunos para que las clases que hereden los usen.

La Figura 7 corresponde a un diagrama de actividad que muestra la selección de una estación.

La Figura 9 corresponde a un diagrama de secuencia que muestra la selección de una estación de origen.

Finalmente, la Figura 10 corresponde a un diagrama de secuencia que muestra la selección de un idioma.

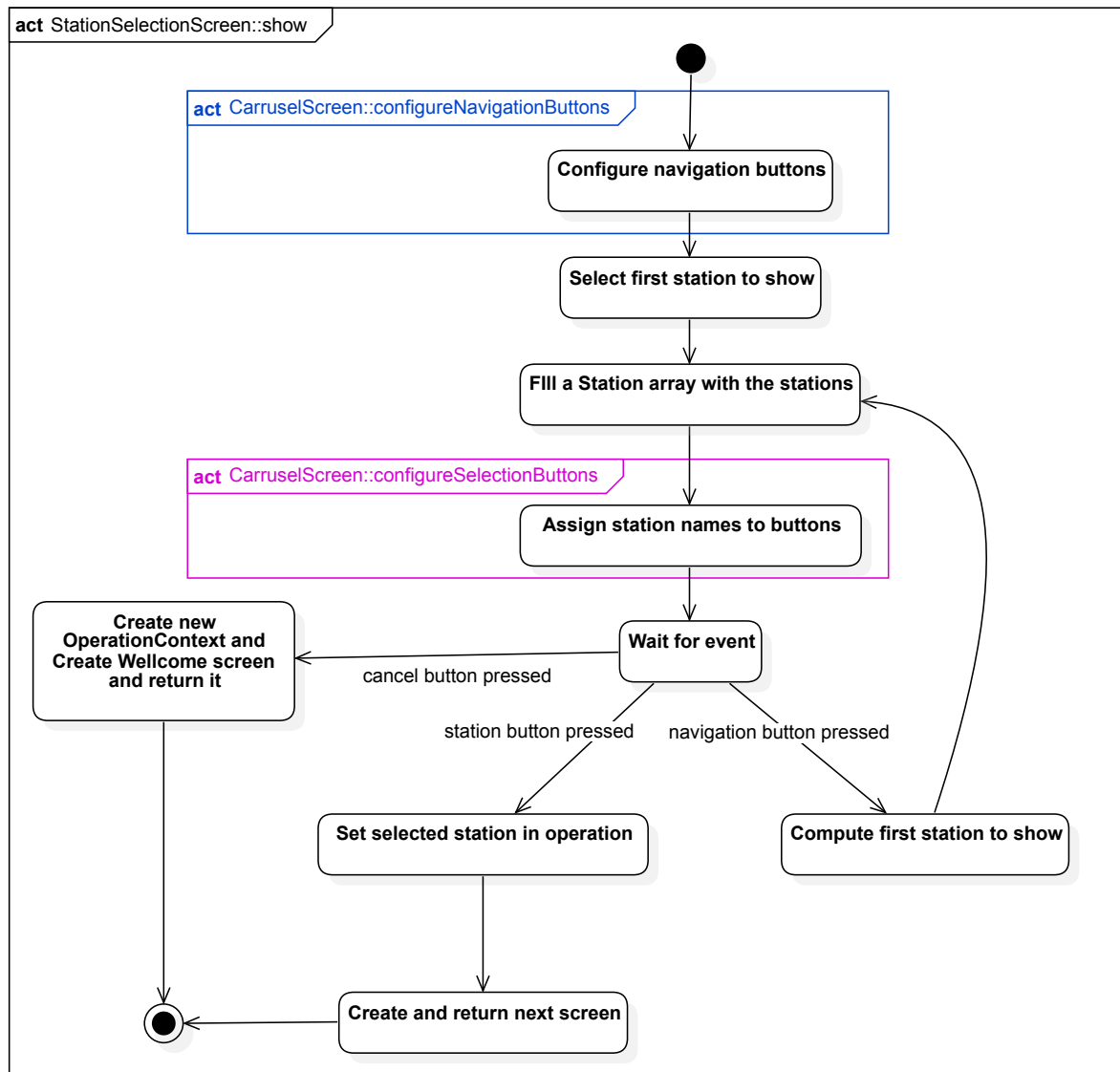


Figura 7.- Diagrama de actividad que refleja el funcionamiento de la pantalla de selección de estación.

Las Figuras 9 y 10 también muestran el funcionamiento del sistema de traducción para cada frase que se pide mostrar a `SelfOrderKiosk`.

Para que se pueda realizar la traducción a otro idioma de las frases que tiene el código, por cada idioma deberá existir un fichero de texto (por ejemplo, la Figura 8 presenta el fichero de traducción de cadenas a inglés). Además, al inicio (ver Figura 6) el `TranslatorManager` creará un objeto `Translator` por cada uno de estos ficheros que se encuentren en el directorio “dictionaries”.

"Cancelar"	"Cancel"
"El precio de la compra es"	"The order price is"
...	...

Figura 8.- Ejemplo de traducción de frases del programa de español a inglés.

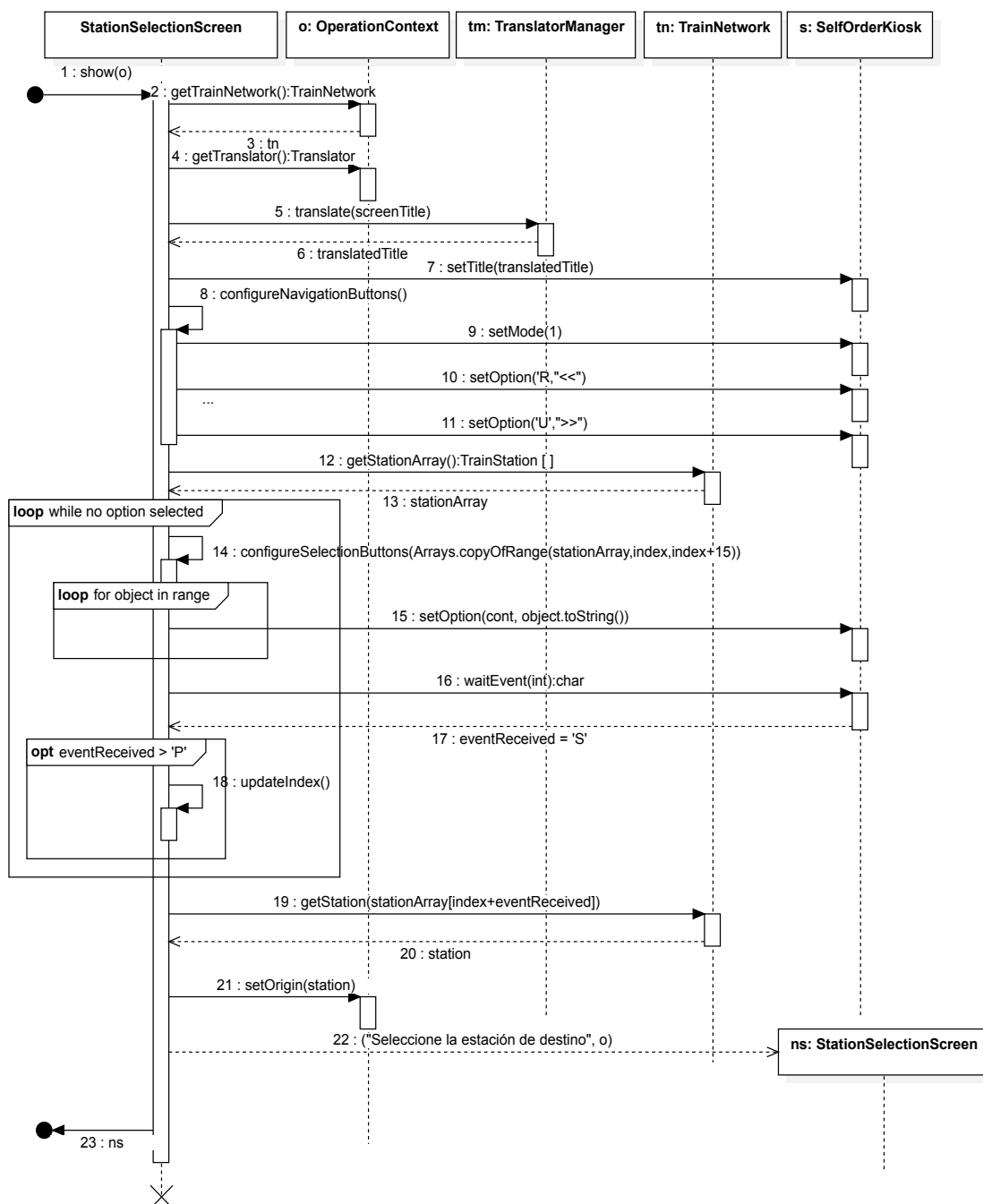


Figura 9.- Diagrama de secuencia que refleja la selección de una estación de origen.

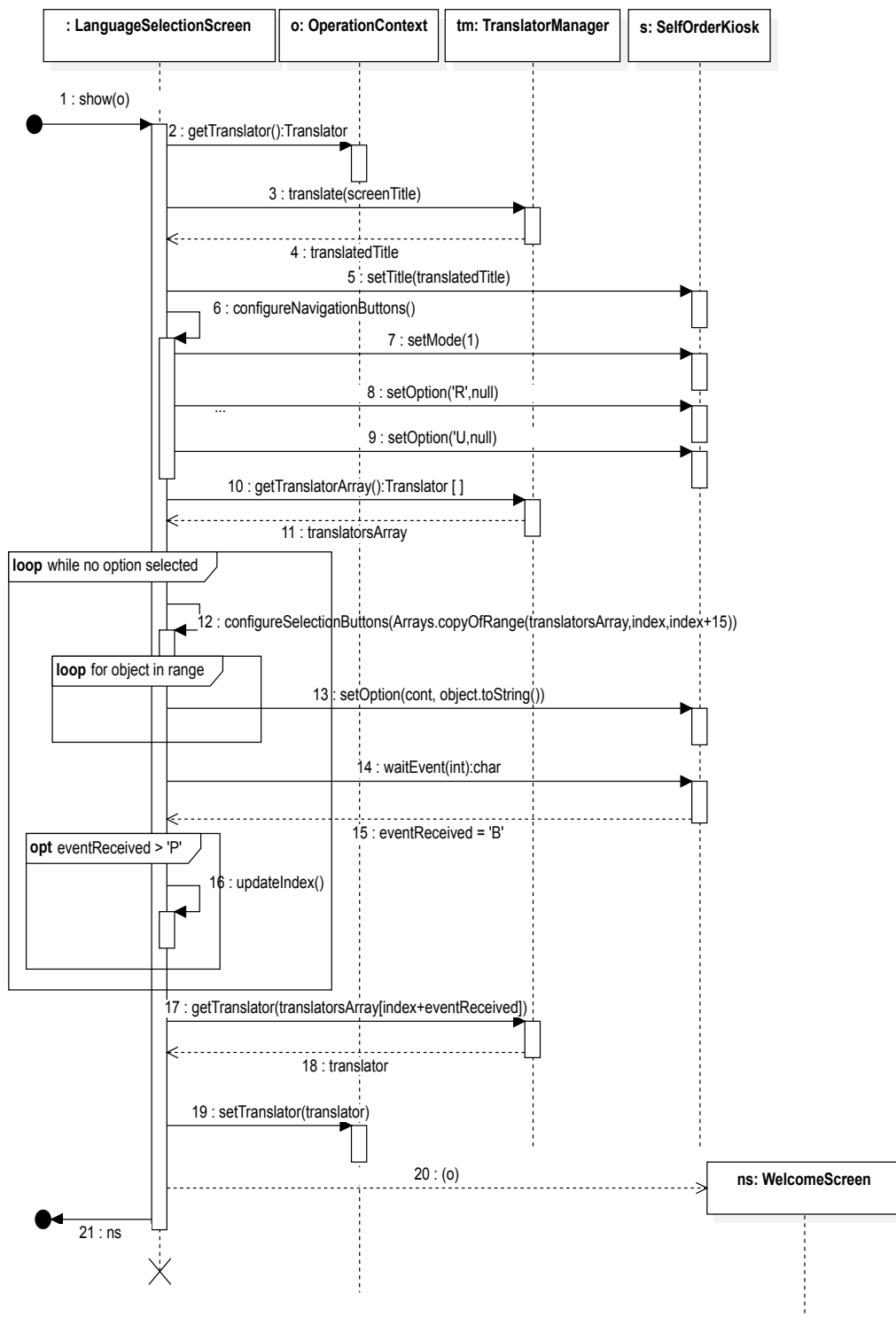


Figura 10.- Pantalla de tipo carrusel para la selección del idioma.

3.3 Pantalla de pago

El diagrama de la Figura 11 muestra el funcionamiento de la pantalla de pago.

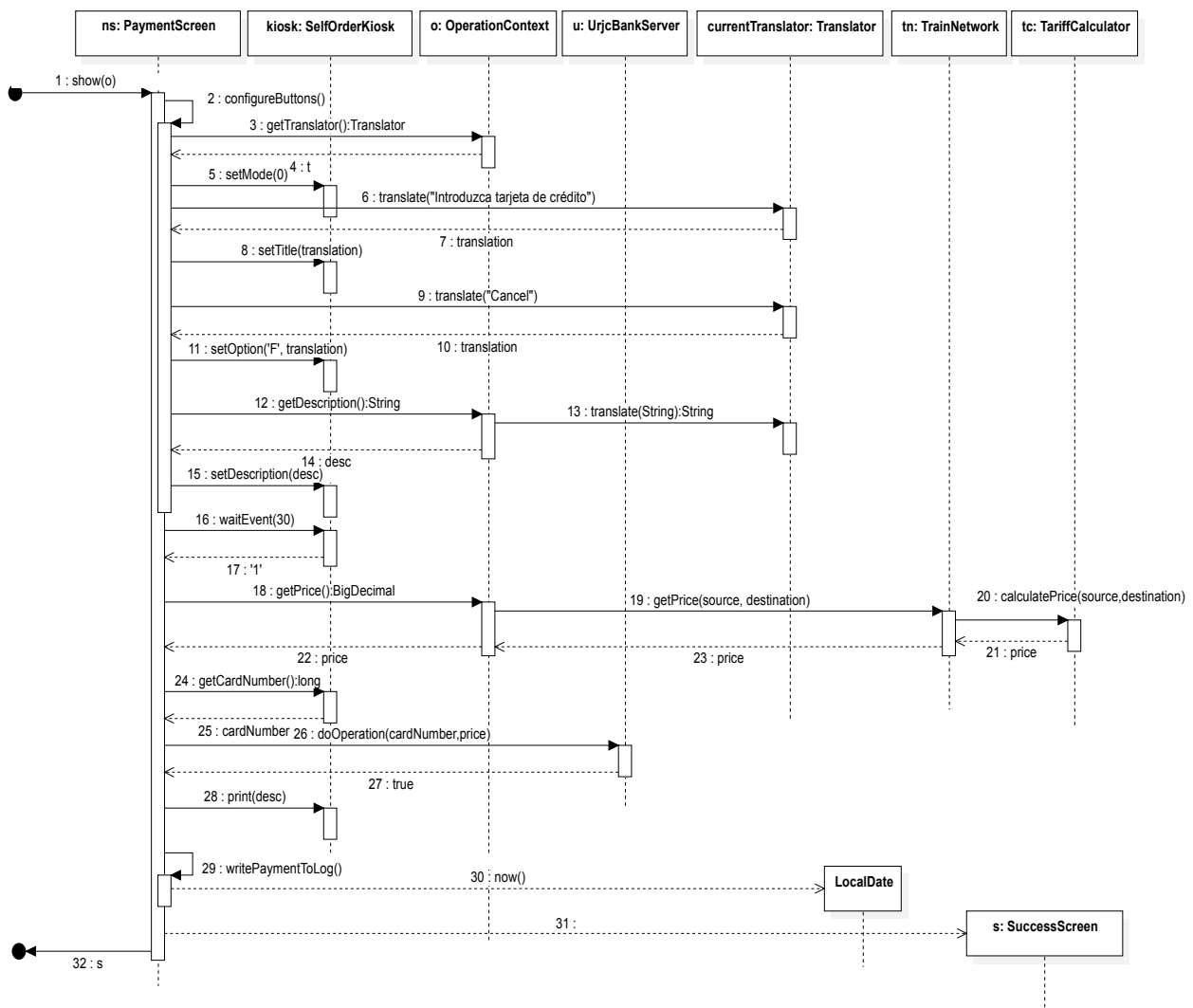


Figura 11.- Gestión de la pantalla de pago.

4 Conclusiones

Este documento presenta un diseño orientativo de la práctica del Sistema Automático de Venta de Billetes de Cercanías. En particular, se han presentado 10 diagramas con numerosas explicaciones que cubren todos los requisitos del documento de ERS. La tabla de la Figura 12 muestra junto al título de cada requisito, el número de aquellos diagramas que tienen alguna relación.

A pesar de todas estas descripciones y diseños, para obtener una aplicación funcionando, hay muchos aspectos necesarios que quedan fuera del alcance de este documento. Dichos aspectos los debe resolver el estudiante en la implementación. Estos aspectos pueden incluir la creación de nuevos métodos e incluso clases, aunque se deben seguir las líneas maestras marcadas en este documento.

Requisito	Diagramas que lo cubren
R1 – Leer las estaciones, y la zona a la que pertenecen, de un fichero	4 7
R2 – Leer los precios de los viajes entre zonas de un fichero	7
R3 – Presentar pantalla de inicio con dos botones	3 7
R4 – Permitir el paso al proceso de compra de billetes	1 5
R5 – Cargar los idiomas disponibles	6 8
R6 – Elegir el idioma para la siguiente compra	8 10
R7 – Presentar en el idioma elegido todos los mensajes	7 9
R8 – Permitir cambiar el idioma por defecto	6
R9 – Presentar estaciones en la pantalla	3 7 9
R10 - Permitir la navegación entre estaciones	3 7 9
R11 – Permitir seleccionar la estación de origen	3 7 9
R12 – Pasar a la pantalla de selección de destino	3 7 9
R13 – Permitir cancelar la operación de selección de estación	1 3 5
R14 – Permitir seleccionar la estación de destino	3 7 9
R15 – Pasar a la pantalla de pago	11
R16 – Permitir cancelar la operación de pago	1 3
R17 – Calcular el precio del billete	4 11
R18 – Presentar resumen de la operación	3 9
R19 – Insertar la tarjeta de crédito y realizar el cobro	3 9
R20 – Imprimir un billete con el resumen de la operación	3 9
R21 – Mostrar pantalla de éxito y retirar tarjeta de crédito	3 9
R22 – Grabar en un fichero el resumen	3 9
R23 – Cada día utilizar un fichero nuevo	11

R24 – En todas las pantallas habrá un botón para cancelar	1 5
R25 – Contemplar el caso de falta de comunicación	1 3 5

Figura 12.- Tabla que muestra la relación de los requisitos con las diferentes figuras.